

Diseño e Implementación de Metaheurísticas para el *Location Routing Problem*

Fuerza Bruta · ALNS · MA|PM — Implementación en C++17

Sebastián Andre Lizárraga Calderón Johannes Albert Loayza Huamán

June 12, 2026

Agenda

- 1 Arquitectura del Software
- 2 Algoritmo de Fuerza Bruta
- 3 ALNS: Adaptive Large Neighborhood Search
- 4 MA|PM: Algoritmo Memético
- 5 Ejemplo Paso a Paso
- 6 Validación contra Fuerza Bruta
- 7 Resultados en Benchmark Prodhon
- 8 Conclusiones

Estructura del Proyecto

Tres bibliotecas CMake:

lrp_core — Dominio

- parser: lee archivos .dat (Prodhon)
- distancia: matriz euclidiana d_{ij}
- estado: EstadoLRP + función objetivo
- solucion_inicial: greedy All-Open

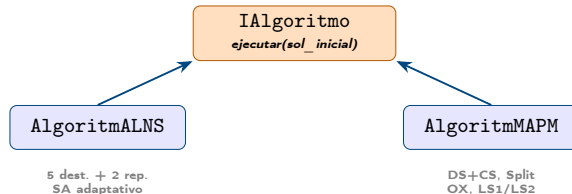
alns_lib

operadores → alns → solver

mapm_lib

cromosoma → split → poblacion →
local_search

Interfaz común IAlgoritmo:



Función objetivo:

$$Z = \underbrace{\sum_{i \in D_{open}} O_i}_{\text{apertura}} + \underbrace{\sum_r (F + \text{dist}(r))}_{\text{rutas}} + 10^6 \cdot |\text{no-asig}|$$

Tipos Principales

EstadoLRP — Solución

- rutas: `map<depot_id, vector<Ruta>`
- depositos_abiertos: conjunto de IDs activos
- no_asignados: clientes sin ruta
- *matriz, *datos: punteros **no-owning**
- `.copy()`: copia profunda de rutas

Convención: clientes $1 \dots n_c$; depósitos $n_c+1 \dots n_c+n_d$. Matriz 0-indexed.

struct Ruta — Circuito de un vehículo

- id_deposito: depósito que origina la ruta
- clientes: `vector<int>` de IDs 1-indexed
- costo: distancia total $+F$ del circuito
- demanda_acumulada: suma de d_j en la ruta

CromosomaLRP (MA|PM)

- DS[0.. $m-1$]: estado de depósitos (0-indexed). Valor > 0 = abierto; 0 = cerrado.
- CS[0.. $n-1$]: permutación de clientes (*giant tour*)
- fitness: costo decodificado por Split

ResultadoAlgoritmo

- mejor_estado: EstadoLRP óptimo hallado
- costo_inicial / costo_final
- iteraciones, historial_mejoras

Fuerza Bruta — Estrategia Exacta para LRP

El LRP tiene dos decisiones anidadas:

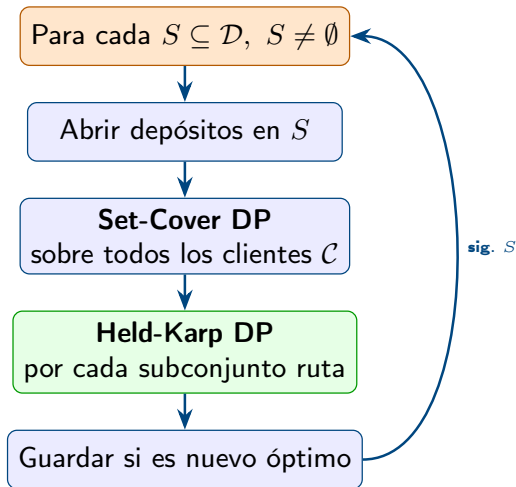
- 1 ¿Qué depósitos abrir? → enumerar los $2^m - 1$ subconjuntos no vacíos
- 2 ¿Cómo partir los clientes en rutas? → dos DP's anidados:

Held-Karp DP Dada una ruta con k clientes, halla el orden óptimo en $\mathcal{O}(2^k \cdot k^2)$

Set-Cover DP Dada la tabla de costos de rutas, halla la partición óptima de clientes en $\mathcal{O}(3^k)$

Complejidad total

$\mathcal{O}(2^m \cdot 3^n)$ — exacto, pero exponencial.
Útil como **validador** para $n \leq 20$.



Held-Karp DP — Orden Óptimo de una Ruta

Algorithm 1 Held-Karp: costo mínimo de ruta

Entrada: clientes $R = \{r_0 \dots r_{k-1}\}$, depósito D , dist. d , costo fijo F

Salida: costo mínimo de recorrer R y volver a D

```
1:  $hk[mask][last] \leftarrow +\infty$  para todo  $(mask, last)$ 
2: for  $i \leftarrow 0$  to  $k-1$  do
3:    $hk[1 \ll i][i] \leftarrow d[D][r_i]$  ▷ ruta de 1 cliente
4: end for
5: for  $mask \leftarrow 1$  to  $2^k - 1$  do
6:   for  $last$  : bit activo en  $mask$ ,  $hk[mask][last] \neq +\infty$  do
7:     for  $j \leftarrow 0$  to  $k-1$ ,  $j \notin mask$  do
8:        $m' \leftarrow mask \mid (1 \ll j)$ 
9:        $hk[m'][j] \leftarrow \min(hk[m'][j], hk[mask][last] + d[r_{last}][r_j])$ 
10:    end for
11:  end for
12: end for
13:  $full \leftarrow 2^k - 1$ 
14: return  $F + \min_{last} (hk[full][last] + d[r_{last}][D])$ 
```

Variables:

mask Máscara de bits: bit i activo $\Leftrightarrow r_i$ ya visitado en el recorrido parcial

last Índice (en R) del último cliente visitado

$hk[mask][last]$ Mínimo costo de recorrer exactamente los clientes en $mask$, terminando en r_{last} (sin retorno al depósito)

F Costo fijo del vehículo, sumado al cerrar

Ejemplo: ruta $\{C_2, C_3\}$ desde D_2

$hk[01][0] = d[D_2][C_2]$

$hk[10][1] = d[D_2][C_3]$

$hk[11][1] = d[D_2][C_2] + d[C_2][C_3]$

Retorno: $hk[11][1] + d[C_3][D_2] + F$

Set-Cover DP — Partición Óptima en Rutas

Algorithm 2 Set-Cover: mínimo costo para cubrir todos los clientes

Entrada: clientes $C = \{c_0 \dots c_{n-1}\}$, depósito D , capacidad Q , dist. d

Salida: costo total mínimo de cubrir C con rutas desde D

```
1: for all subconj.  $T \subseteq C$  con  $\text{demanda}(T) \leq Q$  do
2:    $\text{rc}[T] \leftarrow \text{Held-Karp}(T, D, d)$  ▷ precalcular todas las rutas
3: end for
4:  $\text{dp}[0] \leftarrow 0$ ;  $\text{dp}[\text{mask}] \leftarrow +\infty$  para  $\text{mask} > 0$ 
5: for  $\text{mask} \leftarrow 1$  to  $2^n - 1$  do
6:    $\text{sub} \leftarrow \text{mask}$ 
7:   while  $\text{sub} > 0$  do
8:     if  $\text{demanda}(\text{sub}) \leq Q$  then
9:        $\text{rest} \leftarrow \text{mask} \oplus \text{sub}$ 
10:       $\text{dp}[\text{mask}] \leftarrow \min(\text{dp}[\text{mask}], \text{dp}[\text{rest}] + \text{rc}[\text{sub}])$ 
11:    end if
12:     $\text{sub} \leftarrow (\text{sub} - 1) \& \text{mask}$  ▷ siguiente sub-máscara
13:  end while
14: end for
15: return  $O(D) + \text{dp}[2^n - 1]$ 
```

Variables:

- rc**[T] Costo de Held-Karp del subconjunto T de clientes (incluye F)
- mask** Subconjunto de clientes a cubrir (representado como máscara de bits)
- sub** Posible “última ruta”: cualquier sub-máscara de **mask**
- rest** $\text{mask} \oplus \text{sub}$: clientes ya cubiertos antes de agregar **sub**
- dp**[**mask**] Costo mínimo de cubrir exactamente los clientes de **mask**

¿Por qué 3^n ?

Cada cliente puede estar en **rest**, en **sub** o fuera de **mask**. Son 3 opciones por cliente $\Rightarrow \mathcal{O}(3^n)$.

Algorithm 3 Loop ALNS

Entrada: **solución inicial** s_0 , **parámetros SA** (T_0, α)

Salida: **mejor solución encontrada** s^*

```
1:  $s \leftarrow s_0$ ;  $s^* \leftarrow s_0$ ;  $\mathcal{T} \leftarrow T_0$ 
2: while  $\text{iter} < \text{max\_iter}$  do
3:    $d \leftarrow \text{seleccionar destructor (ruleta por pesos } w_d)$ 
4:    $r \leftarrow \text{seleccionar reparador (ruleta por pesos } w_r)$ 
5:    $s' \leftarrow r(d(s))$ 
6:   if  $f(s') < f(s^*)$  then
7:      $s^* \leftarrow s'$ ;  $s \leftarrow s'$ ; score  $\leftarrow \sigma_1$ 
8:   else if  $f(s') < f(s)$  then
9:      $s \leftarrow s'$ ; score  $\leftarrow \sigma_2$ 
10:  else if  $U(0, 1) < \exp(-\frac{f(s') - f(s)}{\mathcal{T}})$  then
11:     $s \leftarrow s'$ ; score  $\leftarrow \sigma_3$ 
12:  else
13:    score  $\leftarrow \sigma_4$ 
14:  end if
15:   $\pi_d \mathrel{+}= \text{score}$ ;  $\pi_r \mathrel{+}= \text{score}$ 
16:   $\mathcal{T} \leftarrow \alpha \cdot \mathcal{T}$ 
17:  if  $\text{iter} \bmod T_{seg} = 0$  then
18:     $w_i \leftarrow w_i(1 - \rho) + \rho(\pi_i / \theta_i)$  para cada op.  $i$ 
19:    resetear  $\pi_i, \theta_i$ 
20:  end if
21: end while
22: return  $s^*$ 
```

Parámetros de referencia:

Parámetro	Valor
T_0 (temperatura inicial)	50 000
T_{end} (temperatura final)	1
α (enfriamiento)	0.998
ρ (olvido de pesos)	0.1
T_{seg} (segmento adaptivo)	100 iter
σ_1 (nuevo mejor)	33
σ_2 (mejora global)	9
σ_3 (aceptado por SA)	13
σ_4 (rechazado)	1
Iteraciones totales	5 000

La **ruleta adaptativa** ajusta cada T_{seg} iteraciones los pesos de operadores según su rendimiento acumulado.

Algorithm 4 `random_removal(s, q)`

Entrada: solución s , cantidad a remover q

Salida: s con q clientes en no_asignados

```
1: for  $i \leftarrow 1$  to  $q$  do
2:    $c \leftarrow$  cliente aleatorio de alguna ruta de  $s$ 
3:   eliminar  $c$  de su ruta
4:   agregar  $c$  a no_asignados
5:   if ruta queda vacía then
6:     eliminar la ruta (depósito sigue abierto)
7:   end if
8: end for
9: return  $s$ 
```

Algorithm 5 `worst_removal(s, q)`

```
1: for  $i \leftarrow 1$  to  $q$  do
2:    $c^* \leftarrow \arg \max_c$  (costo marginal de  $c$  en su ruta)
3:   eliminar  $c^*$ ; agregar a no_asignados
4: end for
5: return  $s$ 
```

Catálogo completo de destructores:

- `random_removal` Extrae q clientes al azar
- `worst_removal` Extrae los q clientes de mayor costo marginal
- `shaw_removal` Agrupa clientes geográficamente afines (vecindad Shaw)
- `route_removal` Elimina una ruta completa al azar
- `depot_closing` **Decisión de localización:** cierra un depósito y reasigna todos sus clientes

Limitación en LRP

`depot_closing` puede cerrar un depósito, pero los reparadores **no pueden reabrirlo**. Si SA acepta el cierre temprano, el algoritmo queda atrapado en soluciones sub-óptimas.

Algorithm 6 greedy_repair(s)

Entrada: s con clientes en no_asignados

Salida: s con todos los clientes insertados

```
1: while no_asignados  $\neq \emptyset$  do
2:    $\Delta^* \leftarrow +\infty$ ;  $c^*$ ,  $\text{pos}^* \leftarrow \text{nulo}$ 
3:   for all  $c \in \text{no\_asignados}$  do
4:     for all depósito  $d \in \text{depositos\_abiertos}$  do
5:       for all ruta  $r$  de depósito  $d$  do
6:         for all posición  $p$  en  $r$  do
7:            $\Delta \leftarrow \text{costo adicional de insertar } c \text{ en } p$ 
8:           if  $\Delta < \Delta^*$  y  $\text{demanda}(r) + 1 \leq Q$  then
9:              $\Delta^* \leftarrow \Delta$ ;  $c^*$ ,  $\text{pos}^* \leftarrow c, p$ 
10:          end if
11:        end for
12:      end for
13:    end for
14:  end for
15:  Insertar  $c^*$  en  $\text{pos}^*$ ; quitar de no_asignados
16: end while
```

Algorithm 7 regret2_repair(s)

```
1: while no_asignados  $\neq \emptyset$  do
2:   for all  $c \in \text{no\_asignados}$  do
3:      $\Delta_1(c) \leftarrow \text{mejor inserción de } c$ 
4:      $\Delta_2(c) \leftarrow \text{segunda mejor inserción de } c$ 
5:      $\text{arrepentimiento}(c) \leftarrow \Delta_2(c) - \Delta_1(c)$ 
6:   end for
7:    $c^* \leftarrow \arg \max_c \text{arrepentimiento}(c)$ 
8:   Insertar  $c^*$  en su mejor posición
9: end while
```

Diferencia clave

Greedy: inserta siempre el cliente con menor Δ inmediato (miope).

Regret-2: prioriza el cliente que *más perdería* si espera (con visión de futuro).

Representación de un individuo:

DS (Depot State) Vector de m enteros.
 $DS[i] > 0 \Rightarrow$ depósito i abierto; $DS[i] = 0 \Rightarrow$ cerrado.

CS (Client Sequence) Permutación de los n clientes: el *tour gigante* sin indicar depósitos ni rutas.

¿Para qué sirve el Split?

DS solo dice **qué depósitos están abiertos.**

CS dice en qué orden están los clientes.

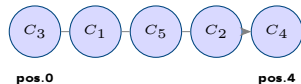
El **Split** es quien determina la partición óptima en rutas y la asignación de clientes a depósitos.

Ejemplo con $n = 5$, $m = 3$:

DS[0]	DS[1]	DS[2]
1	0	1
D_1 abierto	D_2 cerrado	D_3 abierto

$$\text{CS} = [C_3, C_1, C_5, C_2, C_4]$$

← Split asigna a D_1 o D_3 →



El Split corta el CS en segmentos consecutivos y asigna cada segmento al depósito de menor costo.

Algorithm 8 Split: CS $\rightarrow$ rutas óptimas

Entrada: tour gigante CS[0..n-1], depósitos abiertos $\mathcal{D}$, Q , dist. d , costo F

Salida: costo de ruteo (sin apertura de depósitos)

```

1:  $dp[k] \leftarrow +\infty \forall k; dp[0] \leftarrow 0$ 
2:  $prev[k] \leftarrow -1; depot[k] \leftarrow \text{nulo}$ 
3: for  $i \leftarrow 0$  to  $n-1$  do
4:    $cap \leftarrow 0$ 
5:   for  $j \leftarrow i$  to  $n-1$  do
6:      $cap \leftarrow cap + \text{demanda}[CS[j]]$ 
7:     if  $cap > Q$  then break
8:   end if
9:    $mejor \leftarrow +\infty; d^* \leftarrow \text{nulo}$ 
10:  for all depósito  $D \in \mathcal{D}$  do
11:     $c \leftarrow F + d[D][CS[i]] + \dots + d[CS[j]][D]$     ▷ Held-Karp para
12:    CS[ $i..j$ ] desde  $D$ 
13:    if  $c < mejor$  then  $mejor \leftarrow c; d^* \leftarrow D$ 
14:  end for
15:  if  $dp[i] + mejor < dp[j+1]$  then
16:     $dp[j+1] \leftarrow dp[i] + mejor$ 
17:     $prev[j+1] \leftarrow i; depot[j+1] \leftarrow d^*$ 
18:  end if
19: end for
20: end for
21: Reconstruir rutas retrocediendo con  $prev[ ]$ 
22: return  $\sum O(D_i \text{ abiertos}) + dp[n]$ 
    
```

Variables clave:

- $dp[k]$ Costo mínimo de rutear los primeros k clientes del tour CS
- i, j Inicio y fin del segmento actual (posiciones en CS)
- cap Demanda acumulada del segmento $[i..j]$
- $prev[k]$ De dónde vino el mejor corte para llegar a k
- $depot[k]$ Depósito óptimo para el segmento que termina en k

Complejidad

$O(n^2 \cdot m)$: para cada par (i, j) se prueban m depósitos. En la práctica muy rápido porque Q limita el tamaño de los segmentos.

Algorithm 9 LS1: mejora por vecindades (*first-improvement*)

Entrada: solución s (rutas explícitas por depósito)

Salida: s mejorada localmente

```
1: mejoró  $\leftarrow$  verdadero
2: while mejoró do
3:   mejoró  $\leftarrow$  falso
4:   for operador  $op \in \{\text{MOVE, SWAP, 2-OPT}\}$  do
5:     for all par de rutas  $(r_1, r_2)$  (mismo o distinto depósito) do
6:        $\delta \leftarrow$  ganancia de aplicar  $op$  a  $(r_1, r_2)$ 
7:       if  $\delta < 0$  then ▷ mejora de costo
8:         aplicar  $op$ ; recalcular costos
9:         mejoró  $\leftarrow$  verdadero; break
10:      end if
11:    end for
12:    if mejoró then break
13:  end if
14: end for
15: end while
16: return  $s$ 
```

Tres vecindades aplicadas:

- MOVE** Relocalizar un cliente: moverlo a otra posición en la misma ruta o a otra ruta (incluso de otro depósito)
- SWAP** Intercambiar dos clientes de rutas del mismo o distinto depósito
- 2-OPT** Inversión de subsecuencia intra-ruta; reconexión cruzada inter-ruta

LS1 vs. LS2

LS1: aplica los 3 operadores sobre el cromosoma decodificado; actualiza el CS al terminar.

LS2: igual pero aplica 2-opt también entre depósitos distintos (más costoso, menos frecuente).

Algorithm 10 OX Crossover + BPD

Entrada: padres P_1, P_2 (cromosomas); umbral Δ ; población $\mathcal{P}$

Salida: hijo H o rechazo

```
1: Cruce DS: seleccionar aleatoriamente de  $P_1$  o  $P_2$  por posición
2: Elegir puntos de corte aleatorios  $[a, b]$  en CS
3:  $H.CS[a..b] \leftarrow P_1.CS[a..b]$ 
4: Completar  $H.CS$  con el orden de  $P_2$ , omitiendo duplicados
5: Si DS inválido: repair( $H$ )
6: Evaluar  $H$  con Split + LS1
7: for all individuo  $s \in \mathcal{P}$  do
8:    $BPD(H, s) \leftarrow |\text{pares}(H) \triangle \text{pares}(s)|$ 
9:   if  $BPD(H, s) < \Delta$  then
10:    return rechazado
11:   end if
12: end for
13: Reemplazar el individuo de mayor costo en  $\mathcal{P}$  con  $H$ 
14: return  $H$ 
```

▷ demasiado similar

Broken Pairs Distance (BPD):

$$D(T, U) = |\text{pares}(T) \triangle \text{pares}(U)|$$

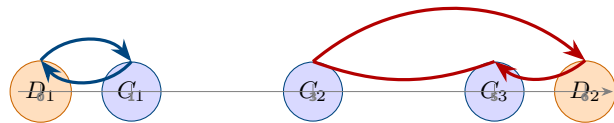
Donde $\text{pares}(T)$ = conjunto de pares (i, j) de clientes adyacentes en alguna ruta de T (par simétrico: clave = $\{\min, \max\}$).

Ciclo de vida de la población

- 1 Inicializar con NN y Clarke-Wright + LS1
- 2 Selección por torneo binario (fitness)
- 3 Cruce OX $\rightarrow$ LS1 $\rightarrow$ BPD check
- 4 Si $NbNoAdd > MaxNbNoAdd$: bajar Δ
- 5 Si $NbAcc > MaxNbAcc$: refrescar población

Ejemplo — Instancia Mínima

5 nodos sobre el eje x :



Matriz de distancias:

	D_1	C_1	C_2	C_3	D_2
D_1	0	1	3	5	6
C_1	1	0	2	4	5
C_2	3	2	0	2	3
C_3	5	4	2	0	1
D_2	6	5	3	1	0

$Q = 2$, $F = 2$, $O = 3$. Demanda de cada cliente = 1.

Ruta azul $D_1 \rightarrow C_1 \rightarrow D_1$: $1 + 1 + F = 4$

Ruta roja $D_2 \rightarrow C_2 \rightarrow C_3 \rightarrow D_2$:

$3 + 2 + 1 + F = 8$

Costo óptimo $= O_1 + O_2 + 4 + 8 = 18$

Los tres algoritmos resuelven esta instancia:

Algoritmo	Costo inicial	Costo final	Observación
Fuerza Bruta	—	18	Garantizado exacto
ALNS	26	18	1ª iter. lo mejora
MA PM	19	18	Split + OX lo encuentran

Ejemplo — Fuerza Bruta paso a paso

Caso A — Solo D_1 abierto:

Partición	Rutas desde D_1	Costo total
$\{C_1, C_2\} + \{C_3\}$	$8 + 12$	$O_1 + 20 = \mathbf{23}$
$\{C_1\} + \{C_2, C_3\}$	$4 + 12$	$O_1 + 16 = \mathbf{19} \checkmark$
$\{C_1, C_3\} + \{C_2\}$	$12 + 8$	$O_1 + 20 = \mathbf{23}$

Set-Cover DP elige $dp[111] = 16$. **Mejor: 19.**

Caso B — Solo D_2 abierto:

Partición	Rutas desde D_2	Costo total
$\{C_1, C_2\} + \{C_3\}$	$12 + 4$	$O_2 + 16 = \mathbf{19} \checkmark$
$\{C_1\} + \{C_2, C_3\}$	$12 + 8$	$O_2 + 20 = 23$

Mejor: 19.

Caso C — D_1 y D_2 abiertos:

Asignación	Costo
$D_1:\{C_1\}, D_2:\{C_2, C_3\}$	$6 + 4 + 8 = \mathbf{18} \checkmark$
$D_1:\{C_2\}, D_2:\{C_1, C_3\}$	$6 + 8 + 12 = 26$
$D_1:\{C_3\}, D_2:\{C_1, C_2\}$	$6 + 12 + 12 = 30$
$D_1:\{C_1, C_2\}, D_2:\{C_3\}$	$6 + 8 + 4 = \mathbf{18} \checkmark$
$D_1:\{C_1, C_3\}, D_2:\{C_2\}$	$6 + 12 + 8 = 26$
$D_1:\{C_2, C_3\}, D_2:\{C_1\}$	$6 + 12 + 12 = 30$

Mejor: 18.

Resultado exacto

$\min(19, 19, 18) = \mathbf{18}$

Dos soluciones óptimas con ambos depósitos abiertos.
La Fuerza Bruta **garantiza** que no existe mejor solución.

Ejemplo ALNS (1/3) — Solución Inicial e Iteración 1

Solución inicial S_0 (All-Open + greedy)

Todos los depósitos abiertos. Greedy asigna C_3 a D_1 (primer procesado).

D_1 : ruta $[C_1, C_3]$: $1+4+5+2 = 12$ D_2 : ruta $[C_2]$:
 $3+3+2 = 8$

$\text{Costo}(S_0) = O_1 + O_2 + 12 + 8 = \mathbf{26}$

Iter. 1 — $\text{random_removal}(q=1)$:

- Se extrae C_3 de la ruta $[C_1, C_3]$ de D_1
- Estado: $D_1:[C_1]$, $D_2:[C_2]$, $\text{no_asig} = \{C_3\}$

greedy_repair: deltas para C_3

Posición	Cálculo Δ	Δ
Tras C_1 en D_1	$4+5-1$	$+8$
Tras C_2 en D_2	$2+1-3$	$0 \checkmark$
Nueva ruta $\{C_3\}$ en D_2	$1+1+2$	$+4$

Resultado: C_3 insertado tras C_2 en D_2
($\Delta=0$):

Ruta	Costo
$D_1 \rightarrow C_1 \rightarrow D_1 + F$	$1+1+2 = 4$
$D_2 \rightarrow C_2 \rightarrow C_3 \rightarrow D_2 + F$	$3+2+1+2 = 8$
$\text{Costo}(S_1) = O_1 + O_2 + 4 + 8$	18

Criterio SA:

$\Delta E = 18 - 26 = -8 < 0 \Rightarrow$ siempre aceptado

S_1 pasa a ser nuevo mejor $S^* = S_1$. Operadores $\text{random_removal} + \text{greedy_repair}$ reciben σ_1 .

Resultado Iter. 1

Inserción con $\Delta=0$: C_3 ya estaba «de vuelta» por esa ruta, agregarlo a D_2 no incrementa recorrido.
De costo 26 a 18 en una sola iteración.

Ejemplo ALNS (2/3) — worst_removal y regret2_repair

Iter. 2 — worst_removal ($q=1$):

- Estado S_1 : $D_1:[C_1]$, $D_2:[C_2, C_3]$
- Se calcula el **ahorro** de extraer cada cliente:

Cliente	Ruta actual	Ahorro (marginal)
C_1	$D_1:[C_1]$ solo	4
C_2	$D_2:[C_2, C_3]$	4
C_3	$D_2:[C_2, C_3]$	0

C_2 : sin él, ruta queda $D_2-C_3-D_2$ (costo 4). Ahorro = $8-4=4$.

C_3 : sin él, ruta queda $D_2-C_2-D_2$ (costo 8). Ahorro = $8-8=0$.

Empate C_1/C_2 (marginal = 4); se elige C_1 .
 $no_asig = \{C_1\}$.

regret2_repair para C_1 :

Opción de inserción	Δ costo	Rango
Nueva ruta $D_1:[C_1]$	+4	Δ_1 (mejor)
Nueva ruta $D_2:[C_1]$	+12	Δ_2 (segunda)
Arrepentimiento	$12-4 = 8$	alto

C_1 vuelve a D_1 . $S_2 = S_1$, costo **18**.

Greedy vs. Regret-2

Greedy: inserta el cliente de menor Δ inmediato (miope).

Regret-2: prioriza el que *más perdería* si espera tu turno. Con un solo no-asignado ambos coinciden.

Ventaja de Regret-2: con **múltiples** no-asignados, inserta primero el de mayor diferencia $\Delta_2 - \Delta_1$, evitando órdenes sub-óptimos.

Ejemplo ALNS (3/3) — depot_closing y Limitación

Iter. 3 — depot_closing:

- 1 Seleccionar depósito a cerrar $\rightarrow$ se elige D_1

- 2 Ahorro por cierre:

$$-O_1 - \text{costo}[D_1:C_1] = -3 - 4 = -7$$

- 3 C_1 debe ir a D_2 . Ruta $[C_2, C_3]$ llena ($Q=2$).

Nueva ruta $D_2:[C_1]$:

$$d(D_2, C_1) + d(C_1, D_2) + F = 5 + 5 + 2 = 12$$

- 4 Costo incremental: $\Delta E = -7 + 12 = +5$

Decisión SA:

$$P = e^{-\Delta E/T_3} = e^{-5/49800} \approx 0.9999$$

$T_3 = 50000 \cdot 0.998^2 \approx 49800$. SA **acepta** (temperatura todavía alta).

S_3 : solo D_2 abierto, rutas $[C_2, C_3] + [C_1]$, costo = **23**.

Limitación: cierre irreversible

Desde S_3 , D_1 **está cerrado** y no puede reabrirse.

Los reparadores (greedy_repair, regret2_repair) solo insertan clientes en depositos_abiertos.

Mínimo alcanzable desde S_3 : costo **19**.

La solución óptima (costo = 18) requiere D_1 abierto.

¿Cómo el ALNS retorna 18?

- La **mejor solución** $S^*=18$ se recuerda siempre
- Conforme $T \rightarrow 0$, SA rechaza retrocesos costosos
- ALNS retorna $S^*=18$ al finalizar las iteraciones
- Mejora futura: operador de *reapertura* de depósitos

Ejemplo MA|PM (1/5) — Población Inicial y Split Chr1

Población inicial

	DS	CS (giant tour)
Chr1	[1, 0]	$[C_2, C_3, C_1]$
Chr2	[0, 1]	$[C_1, C_2, C_3]$

DS[i]=1: dep. i abierto. CS: tour gigante.

DP del Split Chr1:

(DS= [1, 0], solo D_1 , CS= $[C_2, C_3, C_1]$)

$dp[k]$	Valor	Segmento ganador
$dp[0]$	0	inicial
$dp[1]$	8	$[0..0] \{C_2\}$
$dp[2]$	12	$[0..1] \{C_2, C_3\}$
$dp[3]$	16	$[2..2] \{C_1\}: dp[2]+4$

Mejor: $dp[3]=16 < dp[1]+12=20$.

Costos por segmento desde D_1 ($F=2$):

Seg.	Clientes	F +recorrido
$[0..0]$	$\{C_2\}$	$2+3+3 = 8$
$[0..1]$	$\{C_2, C_3\}$	$2+3+2+5 = 12$
$[1..1]$	$\{C_3\}$	$2+5+5 = 12$
$[1..2]$	$\{C_3, C_1\}$	$2+5+4+1 = 12$
$[2..2]$	$\{C_1\}$	$2+1+1 = 4$

Rutas Chr1 (backtracking: $dp[3]=dp[2]+4$, $dp[2]=dp[0]+12$):

- Seg. $[2..2]=\{C_1\}$ en D_1 : costo 4
- Seg. $[0..1]=\{C_2, C_3\}$ en D_1 : costo 12
- Fitness Chr1 = $O_1 + dp[3] = 3 + 16 = 19$

Idea del Split

El Split optimiza la **partición** del CS: para cada segmento prueba todos los depósitos disponibles y elige el de menor costo. Con solo D_1 activo, todas las rutas parten de D_1 .

Ejemplo MA|PM (2/5) — Split Chr2 y Búsqueda Local LS1

Split Chr2: $DS = [0, 1]$ (solo D_2),
 $CS = [C_1, C_2, C_3]$:

Seg.	Clientes	F +recorrido desde D_2
[0..0]	$\{C_1\}$	$2+5+5 = 12$
[0..1]	$\{C_1, C_2\}$	$2+5+2+3 = 12$
[1..1]	$\{C_2\}$	$2+3+3 = 8$
[1..2]	$\{C_2, C_3\}$	$2+3+2+1 = 8$
[2..2]	$\{C_3\}$	$2+1+1 = 4$

$dp[k]$	Valor	Segmento ganador
$dp[0]$	0	inicial
$dp[1]$	12	[0..0] $\{C_1\}$
$dp[2]$	12	[0..1] $\{C_1, C_2\}$
$dp[3]$	16	[2..2] $\{C_3\}$: $dp[2]+4$

Rutas Chr2: $D_2: \{C_1, C_2\} + \{C_3\}$. Fitness
 $= O_2 + 16 = \mathbf{19}$.

Búsqueda Local LS1 sobre ambos cromosomas:

Cromo.	Movimiento	Costo
Chr1	Mover C_2 : $\{C_3\} + \{C_1, C_2\}$	$O_1 + 20 = 23$
	Mover C_3 : $\{C_2\} + \{C_1, C_3\}$	$O_1 + 20 = 23$
	Fusionar (dem. = $3 > Q$)	infactible
Chr2	Mover C_1 : $\{C_2\} + \{C_1, C_3\}$	$O_2 + 20 = 23$
	Mover C_2 : $\{C_1\} + \{C_2, C_3\}$	$O_2 + 20 = 23$

Ambos cromosomas son localmente óptimos: fitness
 $= 19$.

Mínimo local — necesitamos cruce

Ninguna vecindad LS1 puede mejorar individualmente. El **cruce OX** combinará el material genético de Chr1 ($DS = [1, 0]$) y Chr2 ($DS = [0, 1]$) para generar un hijo con nuevas combinaciones y escapar del mínimo local de 19.

Ejemplo MA|PM (3/5) — OX Crossover y Resultado

OX Crossover ($P_1=\text{Chr1}$, $P_2=\text{Chr2}$, corte $[0..0]$):

- 1 Copiar $P_1[0..0]=[C_2]$ al hijo en pos. 0
- 2 Recorrer $P_2=[C_1, C_2, C_3]$, omitir $\{C_2\}$: queda $[C_1, C_3]$
- 3 Hijo.CS = $[C_2, C_1, C_3]$
- 4 Hijo.DS: DS[0] de Chr1= 1; DS[1] de Chr2= 1
 $\Rightarrow$ DS= $[1, 1]$ (ambos depósitos abiertos)

Clave del cruce

Ningún padre tenía DS= $[1, 1]$:

Chr1 tenía $[1, 0]$ y Chr2 tenía $[0, 1]$.

El cruce **combinó ambos estados** y creó un hijo que puede usar los dos depósitos simultáneamente.

Reconstrucción del hijo tras Split ($dp[3]=12$):

- $dp[3]=12$: viene de $dp[2]+4 \rightarrow$ seg. $\{C_3\}$ asignado a D_2
- $dp[2]=8$: viene de $dp[0]+8 \rightarrow$ seg. $\{C_2, C_1\}$ asignado a D_1

Ruta	Costo
$D_1 \rightarrow C_2 \rightarrow C_1 \rightarrow D_1 + F$	$3+2+1+2 = 8$
$D_2 \rightarrow C_3 \rightarrow D_2 + F$	$1+1+2 = 4$
Fitness = O_1+O_2+8+4	= 18 ✓

¿Por qué bajó de 19 a 18?

El hijo heredó DS= $[1, 1]$. El Split pudo asignar $\{C_3\}$ al depósito más cercano (D_2 , distancia 1) en vez del lejano D_1 (distancia 5), reduciendo el costo total en 10.

Ejemplo MA|PM (4/5) — Split del Hijo (DP detallado)

Comparación Chr1 / Chr2 / Hijo:

	Chr1	Chr2	Hijo
DS	[1, 0]	[0, 1]	[1, 1]
CS	$[C_2, C_3, C_1]$	$[C_1, C_2, C_3]$	$[C_2, C_1, C_3]$
Fitness	19	19	18

Split Hijo: DS= [1, 1], CS= $[C_2, C_1, C_3]$ — se prueban **ambos** depósitos por segmento:

Seg.	Clientes	D_1 / D_2 (costo)	dp
[0..0]	$\{C_2\}$	8 / 8 (empate)	$dp[1] = 8$
[0..1]	$\{C_2, C_1\}$	8 / 12	$dp[2] = 8$
[1..1]	$\{C_1\}$	4 / 12	$\min(12, 8) = 8$
[1..2]	$\{C_1, C_3\}$	12 / 12 (empate)	$dp[3] = 20$
[2..2]	$\{C_3\}$	12 / 4	$\min(20, 8+4) = \mathbf{12}$

Ventaja del DS= [1, 1]

Chr1 y Chr2 solo usaban **un** depósito cada uno.

El hijo evalúa **ambos**: el segmento $\{C_3\}$ cuesta 4 desde D_2 vs. 12 desde D_1 .

Costo total: $3+16 = 19 \rightarrow 3+3+12 = \mathbf{18}$.

Ejemplo MA|PM (5/5) — BPD y Convergencia al Óptimo

Broken Pairs Distance (BPD):

$$D(T, U) = |\text{pares}(T) \triangle \text{pares}(U)|$$

$\text{pares}(X)$ = pares de clientes adyacentes en rutas de X
(simétrico: $\{i, j\}$).

Individuo	Pares adyacentes
Chr1	$\{(C_2, C_3)\}$
Chr2	$\{(C_1, C_2)\}$
Hijo	$\{(C_2, C_1)\} = \{(C_1, C_2)\}$

$$D(\text{hijo}, \text{Chr1}) = |\{(C_2, C_3)\} \triangle \{(C_1, C_2)\}| = 2 \geq \Delta=1 \quad \checkmark$$

$$D(\text{hijo}, \text{Chr2}) = |\{(C_1, C_2)\} \triangle \{(C_1, C_2)\}| = 0 < \Delta=1 \quad \times$$

Hijo rechazado por similitud

El hijo (fitness= 18) es **rechazado**: comparte exactamente los mismos pares adyacentes que Chr2.
NbNoAdd += 1

Mecanismo de recuperación

Cuando NbNoAdd supera MaxNbNoAdd, el umbral Δ baja a 0.

Con $\Delta=0$, el hijo se **acepta** y reemplaza al individuo de mayor costo (Chr1 o Chr2, ambos en 19).

MA|PM converge al óptimo 18.

La gestión de diversidad BPD evita convergencia prematura, incluso cuando el mejor hijo encontrado es temporalmente rechazado.

¿Cómo validamos las metaheurísticas?

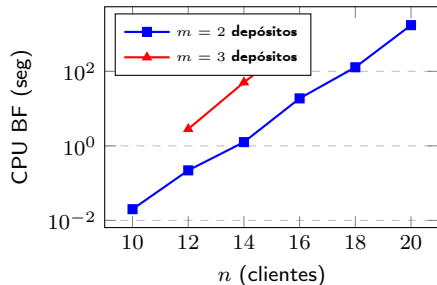
- 1 Construimos instancias propias ($n = 10 \dots 20$, $m = 2$ o 3) con parámetros controlados
- 2 Ejecutamos Fuerza Bruta exacta para obtener el **óptimo comprobado**
- 3 Ejecutamos ALNS y MA|PM sobre las mismas instancias
- 4 Comparamos: $\text{gap} = \frac{Z_{alg} - Z_{BF}}{Z_{BF}} \times 100\%$

Criterio del evaluador

La Fuerza Bruta debe tardar al menos **30 minutos** para demostrar que el problema es computacionalmente intratable y las metaheurísticas son necesarias.

La instancia coord20-2-1 tarda **28.9 minutos**.

Escala de tiempos de la Fuerza Bruta:



Escala logarítmica. El tiempo crece **exponencialmente** con n y m .

Validación — Resultados vs. Óptimo Exacto

Instancia	n	m	Fuerza Bruta		ALNS	MA PM
			Óptimo	CPU (s)	Gap	Gap
coord10-2-1	10	2	11 576	0.02	0.00%	0.00%
coord10-2-2	10	2	15 912	0.02	0.00%	0.00%
coord12-2-1	12	2	15 123	0.22	0.00%	0.00%
coord14-2-1	14	2	17 517	1.26	0.00%	0.00%
coord16-2-1	16	2	30 324	18.78	0.00%	0.00%
coord18-2-1	18	2	31 907	128.40	0.00%	0.00%
coord20-2-1	20	2	35 035	1733.0 (28.9 min)	0.00%	0.00%
coord12-3-1	12	3	17 497	2.83	+5.86%	0.00%
coord14-3-1	14	3	20 288	50.46	+5.82%	0.00%
coord15-3-1	15	3	21 139	191.83	0.00%	0.00%
Resumen			BF exacto, pero exponencial		Gap ≤ 5.9%	Gap = 0%

ALNS gap > 0 en instancias de 3 depósitos: el operador `depot_closing` cierra un depósito que no debería, y los reparadores no pueden reabrirlo.

MA|PM: gap = 0% en **todas** las instancias. Split siempre evalúa la combinación de depósitos abiertos óptima.

¿Por qué existe?

La función objetivo Z puede evaluarse sobre cualquier estado, incluso uno infactible (penalizaciones). El validador responde: *¿cumple el modelo LRP exactamente?*

Restricciones verificadas

R1	Ecs. 2&3	Cada cliente visitado exactamente una vez
R2	Ec. 4	Asignado a un solo depósito
R3	Ec. 5	Rutas solo en depósitos abiertos
R4	Ec. 6	Capacidad de depósito $\leq W_i$
R5	Ec. 11	Capacidad de vehículo $\leq Q$
R6	interna	Coherencia <code>id_deposito</code>

Salida en consola (solución factible):

```
=====
VALIDACIÓN DE RESTRICCIONES LRP
=====
n=10 m=2 | Q=140 F=100
Depósitos abiertos: 11 12   Rutas: 4
-----
R1: [OK] 10 clientes exactamente una vez
R2: [OK] Asignación uno-a-uno garantizada
R3: [OK] Todas las rutas en dep. abiertos
R4: [OK] Dep 11: 320.00 ≤ W=500
R5: [OK] Ruta 1: carga=130.00 ≤ Q=140
-----
Apertura:    4000.00
Costo fijo:   400.00
Costo viaje: 7176.00
=====
STATUS: SOLUCIÓN FACTIBLE Z=11576.00
=====
```

Tipo de retorno: `ResultadoValidacion`

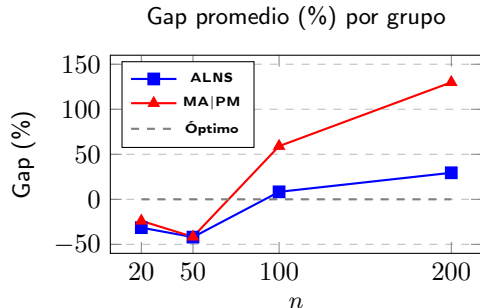
Contiene `factible`, `r1_ok..r6_ok`, listas de violaciones y desglose de costos ($O_i + F \cdot \text{rutas} + \text{viaje}$).

Resultados en Benchmark Prodhon — ALNS vs. MA|PM

Tiempos de CPU (promedio por grupo):

Grupo	Gap ALNS	Gap MA PM
$n = 20$ (4 inst.)	-31.5%	-24.0%
$n = 50$ (3 inst.)	-42.0%	-41.7%
$n = 100$ (12 inst.)	+8.3%	+59.0%
$n = 200$ (6 inst.)	+29.5%	+129.9%

Grupo	CPU ALNS	CPU MA PM
$n = 20$	~ 0.01 s	~ 0.3 s
$n = 50$	~ 0.17 s	~ 4 s
$n = 100$	~ 0.6 s	~ 70 s
$n = 200$	~ 7 s	~ 730 s



Conclusión comparativa

- $n \leq 50$: equivalentes en calidad; ALNS 10× más rápido
- $n = 100$: ALNS gana todas las instancias; $\sim 100\times$ más rápido
- $n = 200$: brecha drástica en ambas métricas
- Gap negativo = **mejor que el BKS registrado**

Lo que implementamos:

- **Fuerza Bruta:** Held-Karp DP + Set-Cover DP, exacto para $n \leq 20$
- **ALNS:** 5 destructores + 2 reparadores, SA + ruleta adaptativa
- **MA|PM:** cromosoma DS+CS, Split $\mathcal{O}(n^2)$, NN/CW, OX, LS1/LS2
- Arquitectura modular C++17 con interfaz común IAlgoritmo

Validación rigurosa:

- BF confirma el óptimo exacto hasta $n = 20$, $m = 3$
- BF en coord20-2-1: **28.9 min** $\Rightarrow$ justifica metaheurísticas
- MA|PM: gap = 0% en todas las instancias validadas
- ALNS: gap $\leq 5.9\%$ (limitación conocida por depot_closing)

Hallazgos experimentales (Prodhon):

- $n \leq 50$: ambos algoritmos superan el BKS registrado
- $n \geq 100$: ALNS domina en calidad y tiempo
- MA|PM escala mal: $\sim 100\times$ más lento que ALNS en $n = 100$

Trabajo futuro

Diseñar un algoritmo propio que combine la velocidad del ALNS con la robustez del MA|PM para la decisión de apertura de depósitos.